O projeto que vamos construir

detectar, investigar, corrigir, verificar, com o humano liberando no meio

O DESTINO

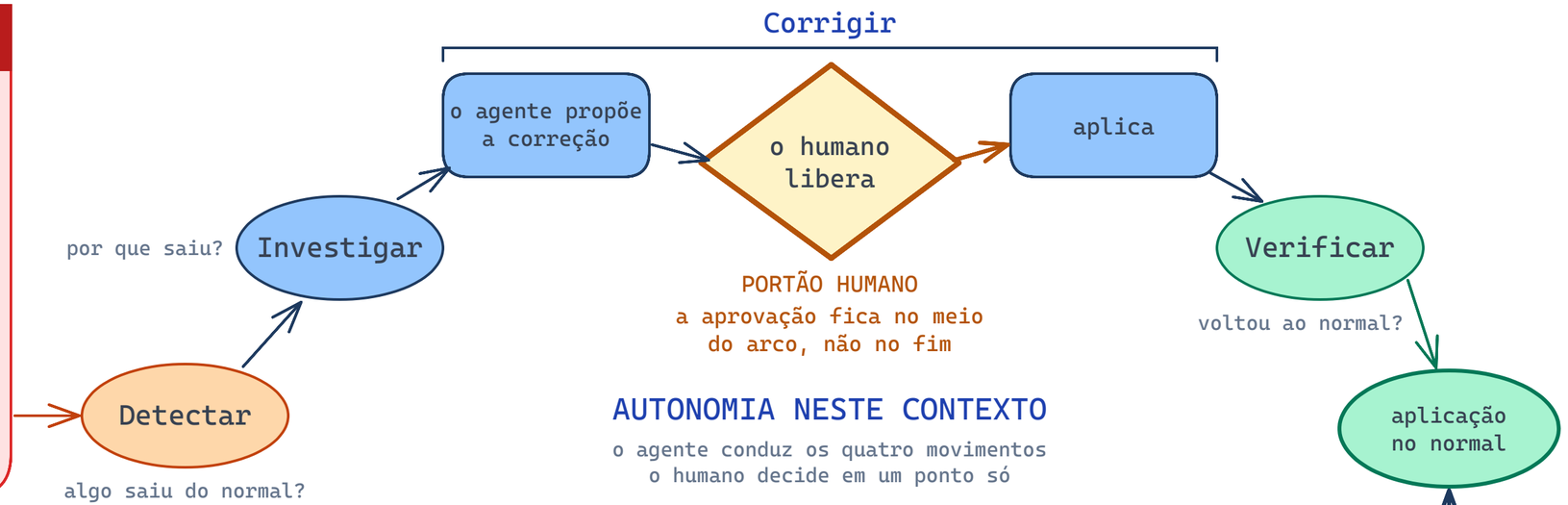
INCIDENTE DE DOMINGO

O QUE
o incidente de domingo

QUANDO
domingo, ao vivo

CONDIÇÃO DE VITÓRIA
a aplicação volta ao normal

REGRA
ninguém toca em nada para resolver



AUTONOMIA NESTE CONTEXTO

o agente conduz os quatro movimentos
o humano decide em um ponto só

O CAMINHO ATÉ LÁ

o que se leva de cada turno

Sábado de manhã

- os fundamentos e o mapa das oportunidades
- aplicação no ar, no Compose e no cluster

Sábado à tarde

- duas skills: Docker e Kubernetes
- o mesmo trabalho da manhã refeito com harness
- incidente corrigido

Domingo

- a especificação
- o agente construído a partir dela

chega ao destino

Recados importantes

o que fazer · quando chega · como usar o chat · onde roda

ação sua · entrega para você

1 — ÚLTIMA CHANCE — Hoje, até 12:00
comprar a gravação

2 — AÇÃO — Dia 01, tarde
assinar a lista

3 — AÇÃO — Dia 02, tarde
assinar a lista

4 — ENTREGA — Até 23/10
certificado emitido e enviado

5 — ENTREGA — Até 26/10
gravação na Kiwify

Certificado

emitido pela Faculdade Full Cycle

Dia 02

Lista de presença · Dia 01

Nome	Assinatura
seu nome	ok
...	
...	

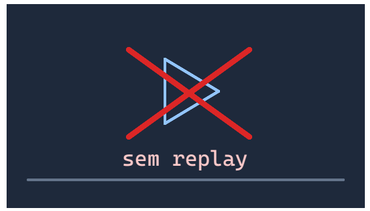
2 3 a lista circula à tarde nos dois dias
(o instrutor relembra na hora)

sem assinatura nos dois dias ⇒ sem certificado

4 emissão e envio até 23/10

Gravação

o evento não tem replay



1 quer a gravação?

janela de compra aberta hoje, até 12:00

comprou

recebe até 26/10 na Kiwify (5)

Dúvidas e chat

três estados ao longo do evento

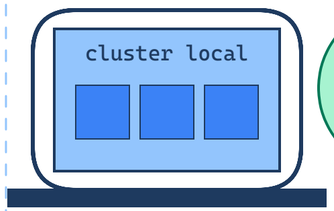
Durante a explicação
chat desligado — off
dispersa quem está acompanhando na máquina

Durante o evento
só dúvidas de conteúdo
dúvida de conteúdo — ok

No final
espaço aberto para as demais dúvidas

Onde isso roda

tudo na sua máquina



sem conta de cloud sem custo sem quota

máquina do aluno

Kubernetes é agnóstico: o que se constrói aqui vale lá

cluster gerenciado

o que muda fora do cluster local: fim de domingo, na correção

Patrocínio: Pós-Graduação em AIOps e Engenharia Cloud

Forward Deployed Engineer: quatro funções recombinadas em torno do resultado

não é função nova saída do nada: é a recombinação de quatro funções adjacentes, cada uma com uma lacuna

De onde vem

quatro funções adjacentes, cada uma com o que falta

Software Engineer

⇒ plataforma central, sem o resultado de um cliente específico

Solution Architect

⇒ orienta e desenha, pode não operar em produção

Consultor técnico

⇒ termina na recomendação, sem ciclo de volta para o produto

DevOps/SRE

⇒ plataforma interna, não a interseção com o problema do cliente



O que faz

cinco movimentos que fecham um ciclo

1. entrar no contexto do cliente e entender o workflow real
2. converter requisito incompleto em escopo e arquitetura
3. construir e fazer deploy de software de produção
4. medir adoção e resultado operacional, não entrega de código
5. transformar aprendizado particular em componente reutilizável

o aprendizado volta para o produto

Onde já aparece no Brasil

o perfil já existe, com e sem esse nome

O título exato já está no país

Google Cloud (São Paulo)

HappyRobot

Nuvia

Wonderful

CloudHive

O mesmo trabalho sob outros nomes

Applied AI Engineer (OpenAI, São Paulo)

Deep Learning Architect, GenAI (AWS)

Lead AI Engineer, Productivity Systems (Nubank)

AI Engineer Sênior, GenAI e Agentes (Itaú)

não é só fornecedor de modelo: banco, fintech e marketplace já contratam

O que não dá para afirmar

- quantas vagas existem
- qual a taxa de crescimento

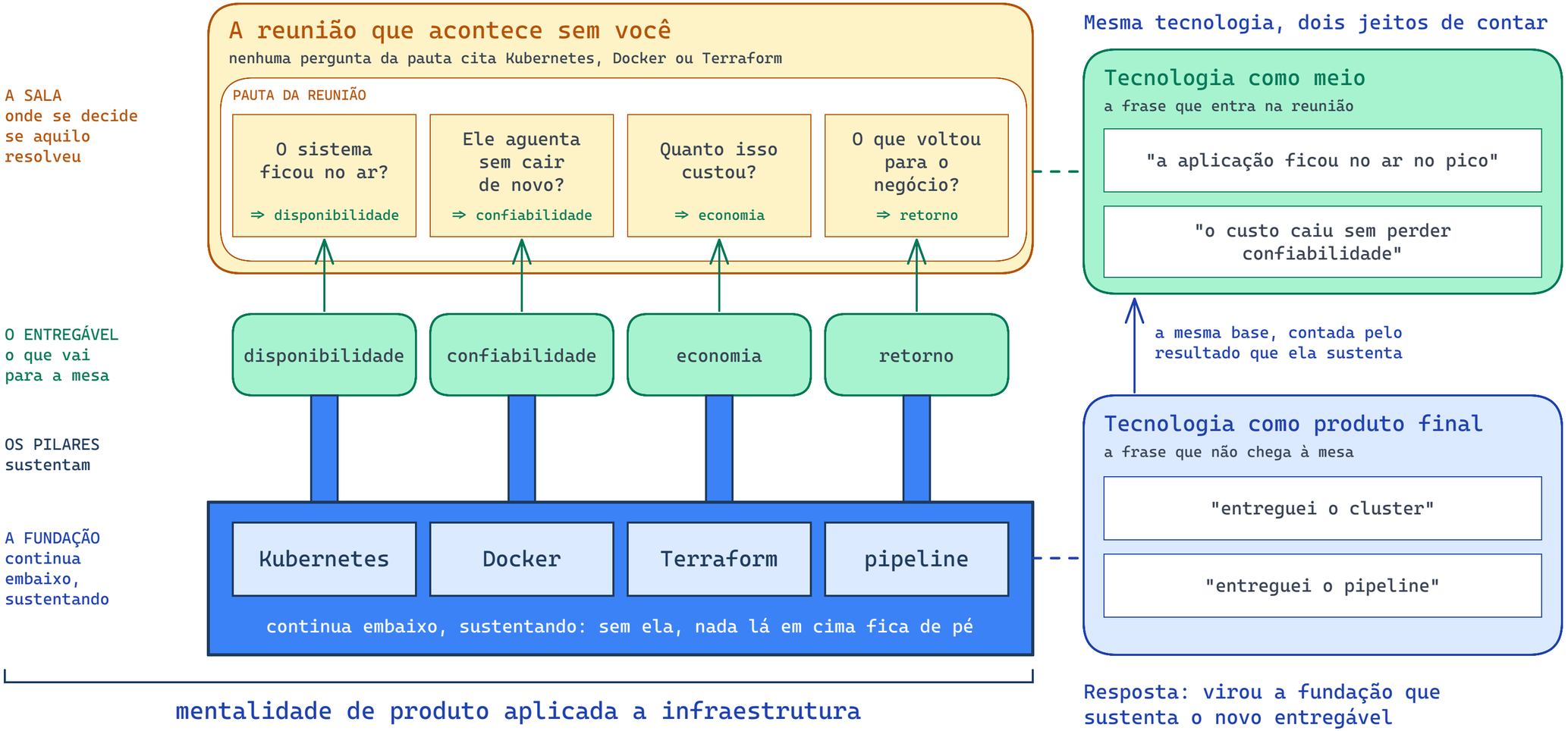
A TRAVA

Não substitui DevOps como disciplina · substitui DevOps como teto de carreira

a base continua sendo o que permite avaliar a IA e impor limite ao agente

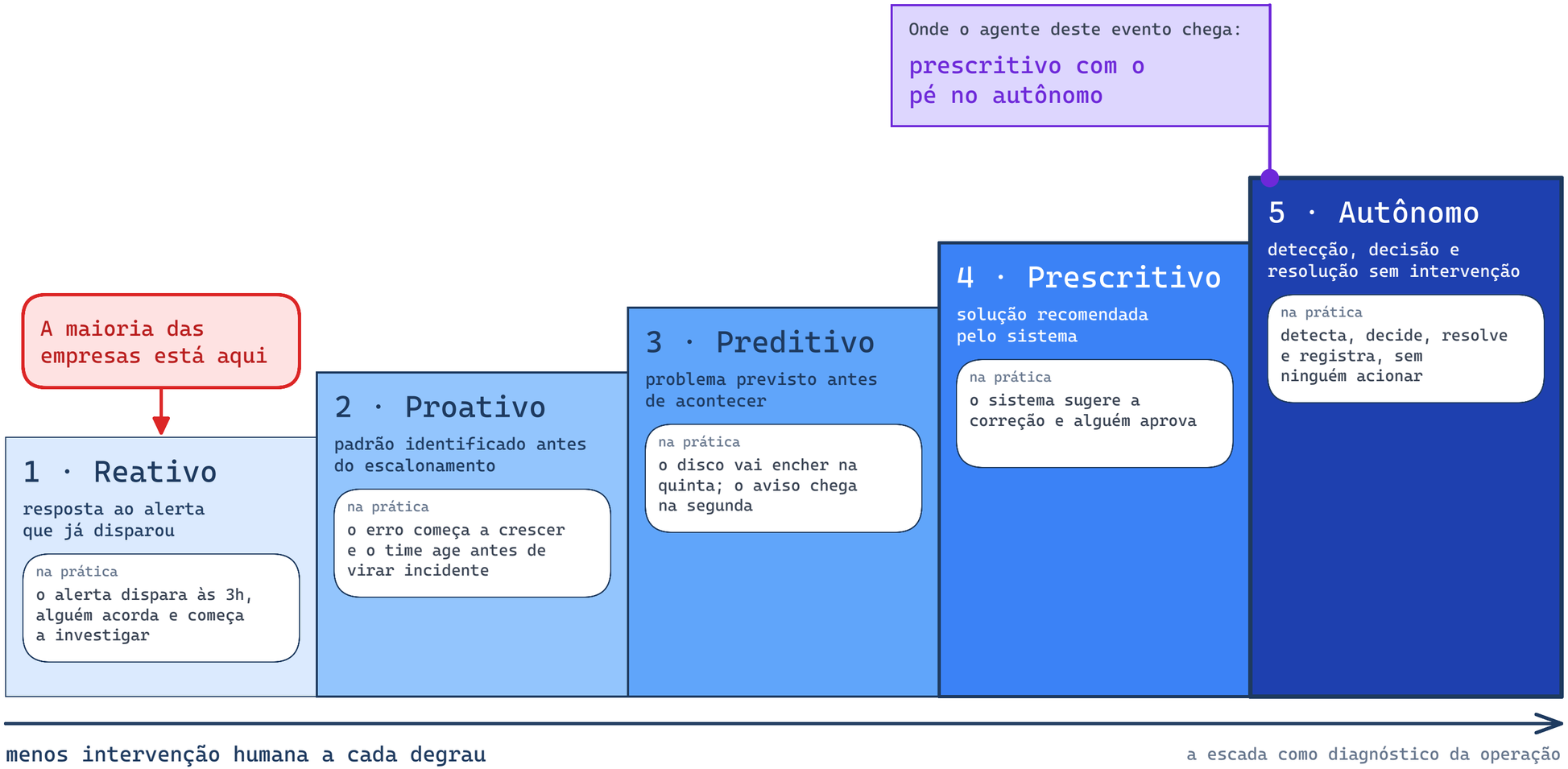
A sua entrega mudou

Se o entregável deixou de ser o cluster e o pipeline, o que aconteceu com tudo que você aprendeu para construir os dois?



A escada de maturidade de AIOps

Em qual desses cinco degraus a sua operação está hoje? De verdade, não onde ela queria estar.

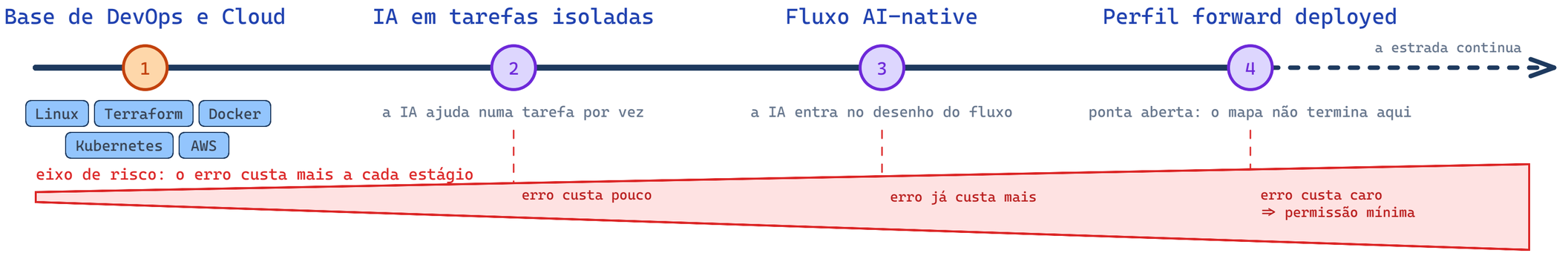


O mapa AI-first

A escada mede a operação · o mapa mede você (dois eixos: um não continua o outro)

AI-first é critério de desenho: considerar a IA ao desenhar o fluxo, não plugar um chatbot no fim

O que impede você de começar hoje, sem esperar a empresa abrir um projeto de IA?



autonomia da IA: observa ⇒ recomenda ⇒ executa sob critério

pergunte em cada estágio: ● o que acontece se a IA errar? ● quando avaliar a resposta? ● como começar com baixo custo, pouca permissão e resultado fácil de verificar?

O próprio fluxo como território

as etapas que você já opera hoje são o lugar de aplicar



em cada etapa, pergunte:

- onde tem trabalho repetitivo?
- onde se perde tempo procurando contexto?
- onde uma análise demora?
- o que a IA pode gerar, executar ou avaliar aqui?

Por onde começar

sem esperar a empresa

terraform plan analisado sem executar o apply

```
$ terraform plan
Plan: 3 to add
⇒ lido, apply não roda
```

falha de pipeline investigada, correção preparada

```
job build: failed
⇒ causa investigada
⇒ correção preparada
```

log correlacionado com mudança recente

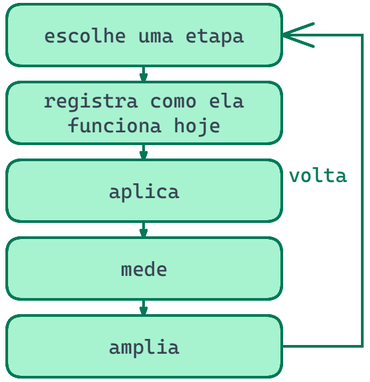
```
13:58 deploy v42
14:02 ERROR 500
⇒ mesma janela
```

anomalia de custo identificada

```
custo/dia fora do padrão
⇒ recurso apontado
⇒ sem alterar nada
```

Ciclo de evolução

começa pequeno e volta



Uma chamada ao modelo

você pede, o seu código chama a API, o modelo responde: cada volta custa token

Loop
Harness
Contexto
Prompt

O QUE VOCÊ CONTROLA

Você pede

o seu código monta a chamada

a chamada leva o prompt

API invoke(prompt)

token de entrada

O QUE VOCÊ NÃO CONTROLA

modelo

resposta

token de saída

a resposta volta pela API

```
{"servico": "pagamentos-api",
 "severidade": "alta",
 "mensagem": "503 sustentado por 3 minutos"}
```

o prompt

```
Transforme este alerta em uma linha JSON com
os campos servico, severidade e mensagem.

Alerta: pagamentos-api respondendo 503 há 3 minutos
```

o prompt é o que você controla aqui

cada chamada é independente

a próxima invoke chega sem esta; o modelo responde só ao que vem dentro da chamada

Engenharia de prompt

O que é ⇒ escolher como pedir

O que decide ⇒ a natureza da tarefa — não a preferência de quem escreve

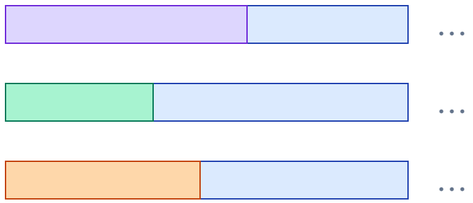
transformação de formato
pedido direto: zero-shot

×

diagnóstico
raciocínio explícito

Quanto dura ⇒ morre com a mensagem

custo desta chamada



cada volta cobra de novo: entrada + saída

Zero-shot × few-shot

mesma tarefa, mesmo modelo, mesmo incidente: muda só o bloco de exemplos

entrada comum

o mesmo incidente nas duas colunas

```
Anotações do plantão: Squad Cobrança, 2026-08-19
- 01:50 fila-cobrancas reentregando mensagem já processada
- Terraform da fila aplicado 01:38: ack timeout caiu de 5 min para 30s
- cada lote leva uns 90s ⇒ todo lote estoura e volta pra fila
- 1.284 cobranças duplicadas · plantão: @carla.ops
```

ZERO-SHOT — sem exemplos

prompt

```
Você trabalha no time de operações e documenta
incidentes já encerrados como post-mortem.
Documente o incidente abaixo como post-mortem.

+ anotações do plantão
```

nenhum exemplo

saída — bom post-mortem, outro padrão

```
# Post-mortem: duplicidade de cobranças
## Impacto
1.284 cobranças duplicadas no processamento.
## Lições aprendidas
Recomenda-se criar um alerta de taxa de reentrega.
```

formato de mercado não o do time

FEW-SHOT — 2 exemplos no prompt

prompt

```
Você trabalha no time de operações e documenta
incidentes já encerrados como post-mortem.
Documente o incidente abaixo como post-mortem.
Siga o padrão dos dois post-mortems abaixo.
+ anotações do plantão
```

exemplo 1: certificado

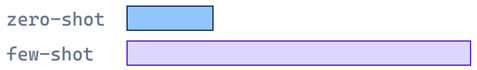
```
### Incidente: Falha na entrega de webhooks ...
**Data:** 2026-06-03 · **Duração:** 25 minutos
**Severidade:** P2 · **Responsável:** @ana.sre
**Ações corretivas:**
- [ ] Criar alerta de certificado ... — @ana.sre — até 2026-06-10
+ exemplo 2: disco cheio no nó
```

saída — o padrão do time

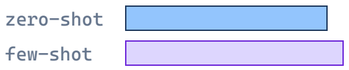
copia o formato

```
### Incidente: Cobranças duplicadas por ack timeout na fila-cobrancas
**Data:** 2026-08-19 · **Responsável:** @carla.ops
**Ações corretivas:**
- [ ] Criar alerta de taxa de reentrega — @carla.ops — até 2026-08-26
```

entrada (tamanho do prompt)



saída



few-shot paga na entrada, em toda chamada

régua

padrão da empresa ⇒ few-shot
padrão do mercado ⇒ zero-shot resolve

como escolher os exemplos

- dois exemplos
- causas diferentes (certificado, disco)
- o mais completo por último
- o exemplo nunca é o alvo

Chain-of-thought

o raciocínio explícito como recurso e como custo

entrada: a evidência

```
# log da pagamentos-api
03:12:41 WARN  timeout acquiring connection from pool (waited 30s)
03:12:41 ERROR POST /v1/cobrancas → 503

# métricas do container
memória do container    estável em 380Mi de 1Gi
CPU do container        estável em 22% do limite

# banco
db-financeiro: conexões ativas 38 → 100, cravado em 100 há 40 min

# eventos
01:05 deploy pagamentos-api v2.14.0: workers de 4 para 12
faturamento-worker com erro de conexão, sem deploy há 11 dias
```

hipóteses concorrentes no mesmo recorte

memória · CPU · banco · deploy

prompt: 4 etapas nomeadas

Você trabalha no time de operações e investiga incidentes em produção.

Antes de concluir qualquer coisa, percorra o raciocínio em voz alta, nesta ordem:

1. O que você observa
2. O que isso descarta
3. O que sobra como hipótese
4. O que checar para confirmar

Só depois de percorrer as quatro etapas, escreva a conclusão em uma frase.

cada etapa nomeada vira um trecho da saída

saída: o percurso afunila

observa
503 intermitente, timeout no pool, banco cravado em 100

descarta
~~memória~~ e ~~CPU~~ (estáveis)

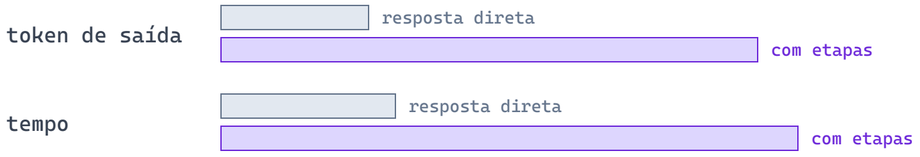
sobra
deploy com 12 workers esgotando o banco compartilhado (e derrubando o faturamento-worker junto)

checar
o limite de conexões do banco

"esse dado eu não tenho"

termina pedindo o que não está na evidência

custo: o preço aparece na saída e no tempo



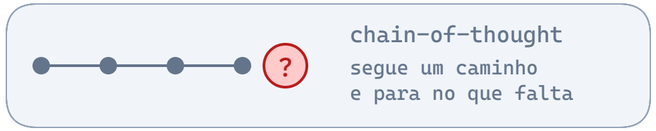
num diagnóstico
gasto
ganho
descarta, afunila, aponta o que falta

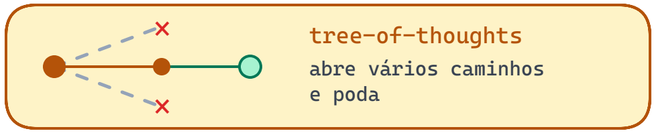
numa tarefa de formato
gasto
ganho
mesmo gasto, nenhum ganho

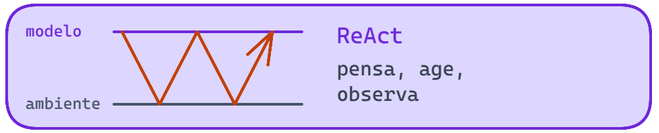
riscos: mais token · mais latência

Ramificar ou agir

duas saídas do raciocínio em linha reta

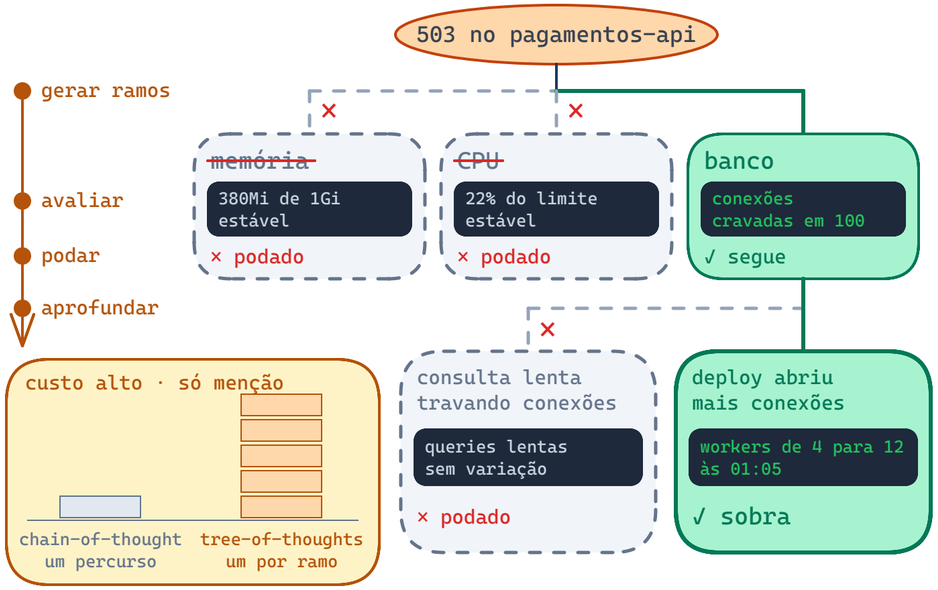






tree-of-thoughts

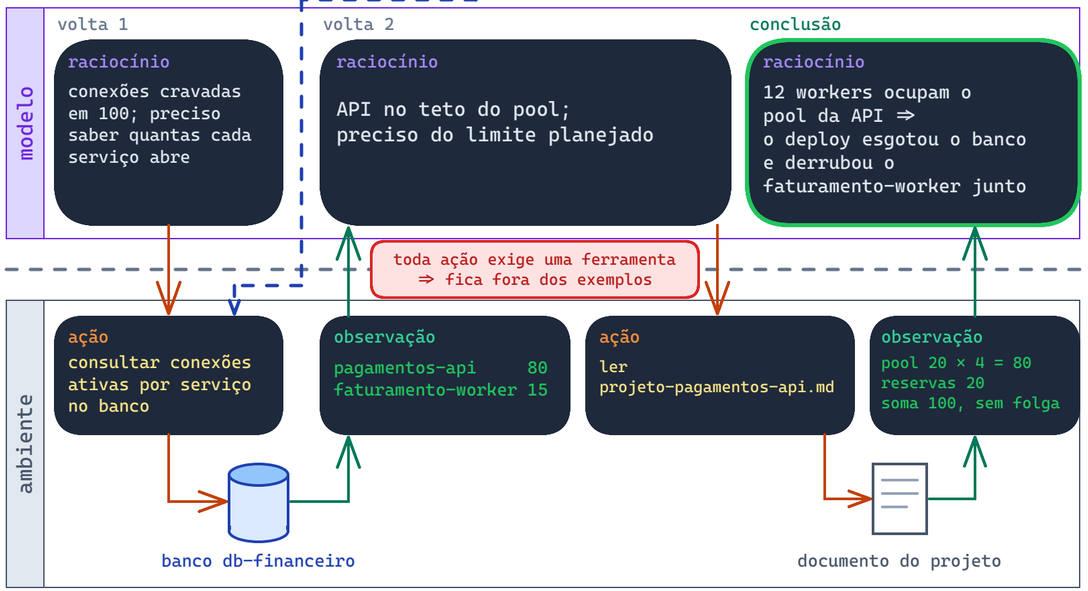
hipóteses em paralelo, com poda



ReAct

raciocínio e ação alternados

chain-of-thought parou em "esse dado eu não tenho"
⇒ ReAct vai buscar



	chain-of-thought	tree-of-thoughts	ReAct
o que faz	só raciocina, um caminho	só raciocina, vários caminhos	raciocina e age no ambiente
onde pesa	saída e tempo	um percurso por ramo	uma chamada por volta + ferramentas
no evento	demo	só menção	é o padrão do agente

Engenharia de contexto

dar para a IA a informação certa, no volume certo, no momento certo



Informação certa

o que esta tarefa precisa saber?

✓ a conta de conexões do projeto-pagamentos-api.md
max_connections 100 · pool 20 × 4 · reservas 20

✕ post-mortem do antifraude — outro problema

✓ o log do horário do erro

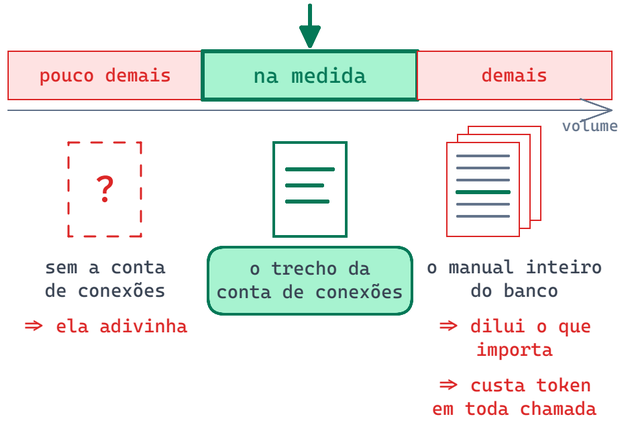
✕ o histórico de outra conversa — outro assunto

filtro: a tarefa

entra na chamada:
✓ a conta de conexões
✓ o log do horário do erro

Volume certo

quanto disso cabe sem atrapalhar?



Momento certo

quando isso precisa estar lá?

- **investigar** ⇒ os eventos e o log do pagamentos-api
- **diagnosticar** ⇒ a conta de conexões do documento de projeto
- **corrigir** ⇒ o procedimento de rollback do projeto
- **verificar** ⇒ o serviço respondendo 200 de novo

✕ o rollback durante investigar ⇒ ruído
a mesma informação, fora da fase

Engenharia de contexto

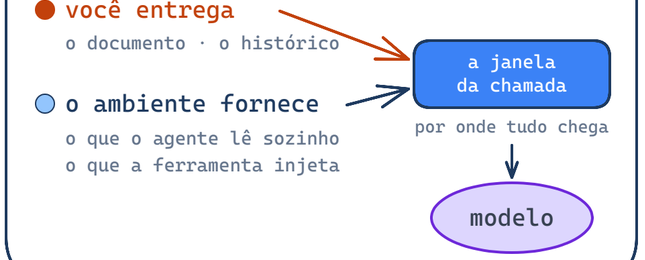
O que é ⇒ informação certa, volume certo, momento certo

O que decide ⇒ o que a IA precisa saber para esta tarefa

- o quê — o log do horário do erro
- quanto — o trecho, não o manual
- quando — na fase de diagnosticar

Quanto dura ⇒ morre com a sessão — o prompt morre com a mensagem

de onde a informação vem

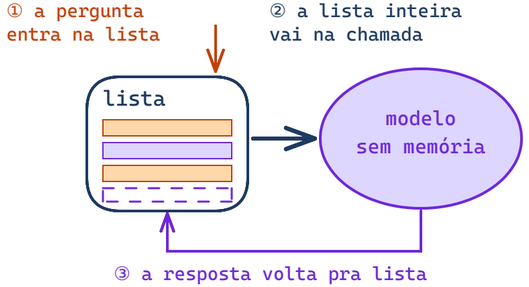


A janela de contexto

uma lista que vai inteira a cada chamada, com tamanho fixo

o ciclo da memória

quem lembra é a lista, não o modelo




```
conversa.append(HumanMessage(pergunta))
resposta = model.invoke(conversa)
conversa.append(resposta)
```

limpar ⇒ a lista some, e a memória junto

a janela desta chamada

borda fixa

tudo é texto: o documento entra como uma mensagem a mais

[1] documento você anexa	projeto-pagamentos-api.md max_connections 100 · pool 20 × 4 = 80 reservas 20 · soma 100, sem folga
[2] usuário histórico	O pagamentos-api está com 503, por onde começo?
[3] modelo histórico	Comece pelo log da aplicação e pelas conexões do banco.
[4] usuário pergunta nova	Qual é o limite de conexões do banco do pagamentos-api?

ocupação

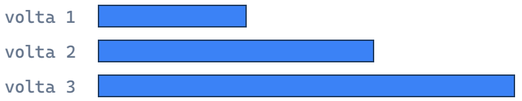
cada token aqui dentro é pago em toda chamada

entra, no volume e no momento certos

	tudo, o tempo todo	na medida, na hora
log da aplicação	uma semana inteira	o recorte do horário do erro, ao investigar
post-mortems	todos, desde a 1ª chamada	o INC-2417 (conexões), quando a hipótese vira banco
histórico	a conversa inteira	um resumo do que já foi descartado

não é se entra: é quanto e quando

contador de entrada

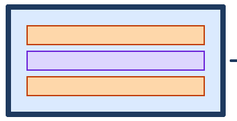


o histórico é pago de novo em toda chamada

---

a mesma pergunta [4], duas escolhas do que entra

sem o documento



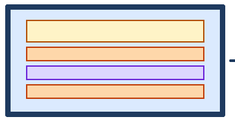
modelo

"Depende da configuração, normalmente ... "

⇒ adivinha

×

com o documento na posição 1



modelo

"max_connections é 100; o pool da API abre até 80 e as reservas fecham em 100.
Fonte: projeto-pagamentos-api.md."

⇒ número e fonte

além da janela, existe:

resumir o histórico	⇒ trocar conversa longa por resumo
janela deslizante	⇒ descartar as mensagens antigas
buscar só o trecho	⇒ trazer o pedaço relevante na hora
memória entre sessões	⇒ guardar fora e recarregar depois
cache do prompt	⇒ não pagar duas vezes o que repete

o que você escolhe mandar na janela muda a resposta

Engenharia de harness

a sessão fecha e leva o que estava nela; o projeto fica e toda sessão nova o encontra

Loop
Harness
Contexto
Prompt

MORRE COM A SESSÃO cada sessão abre, trabalha e fecha levando o que montou

sessão 1
escrever o Dockerfile
~~prompt: "escreve o Dockerfile da API"~~
~~histórico: 12 mensagens~~
~~contexto: saída do docker build~~
fechou ⇒ nada disso volta
leu ao abrir: skill do Dockerfile

sessão 2
subir no cluster
~~prompt: "sobe no cluster"~~
~~histórico: 9 mensagens~~
~~contexto: manifesto e saída do apply~~
fechou ⇒ nada disso volta
leu ao abrir: instruções do projeto + permissões

sessão 3
investigar o 503 do pagamentos-api
~~prompt: "investigue o 503 do pagamentos-api"~~
~~histórico: 15 mensagens~~
~~contexto: eventos e logs do pod~~
fechou ⇒ nada disso volta
leu ao abrir: conexão com o cluster + permissões

skill do Dockerfile
instruções do projeto
permissões
permissões
conexão MCP

SOBREVIVE AO PROJETO tempo →

skill do Dockerfile — instrução
- imagem base enxuta
- usuário sem root
- build separado da execução

instruções do projeto — instrução
- imagens com tag de versão
- nunca latest
  ex.: pagamentos-api:1.4.2

permissões — permissão
ler o cluster: sim
apagar pod: não

conexão com o cluster — ativo · MCP
cluster: kind-imersao
leitura: pods, eventos e logs

Engenharia de harness

O que é ⇒ o que fica escrito e vale para toda sessão

O que decide ⇒ o que o agente sabe e o que pode

Quanto dura ⇒ **sobrevive ao projeto** a sessão morre ao fechar

investigar o 503: sem harness × com harness

○ sem harness

```
> Investigue o 503 do
  pagamentos-api. O cluster é o
  kind-imersao. Use só leitura,
  não apague nenhum pod. Lembre
  que nossas imagens usam tag de
  versão ...
```

⇒ toda sessão reexplica tudo

○ com harness

```
> Investigue o 503 do
  pagamentos-api.
```

kind-imersao vem da conexão MCP
só leitura vem das permissões
tag de versão vem das instruções

⇒ o projeto já diz o resto

O mapa do harness

todo harness responde duas perguntas; cada peça responde uma delas

● neste evento skills e MCP · os demais existem e valem saber

O que o agente sabe?

o que ele encontra escrito ao abrir o projeto

instruções do projeto (rules)

o que vale sempre neste projeto

```
# vale em toda sessão, sem precisar pedir

- imagens com tag de versão, nunca latest
- um namespace por ambiente: dev · prod
- todo manifesto com limite de recurso
- segredo nunca no repositório
```

skills

● neste evento

como fazer do seu jeito, sob demanda

```
skill: dockerfile
# entra só quando o pedido envolve um Dockerfile

Ao escrever um Dockerfile:
- imagem base enxuta
- usuário sem root
- build separado da execução
```

O que o agente pode?

até onde a mão dele alcança, e quem age junto

MCP

● neste evento

o que ele alcança fora do terminal

```
mcp:  ler pods, eventos e logs do cluster
```

permissões

o que pode e o que não pode

```
ler o cluster:  sim  ·  apagar pod:  não
```

hooks

o que roda sempre, sem depender dele

```
antes de aplicar no cluster:  validar o manifesto
```

subagentes

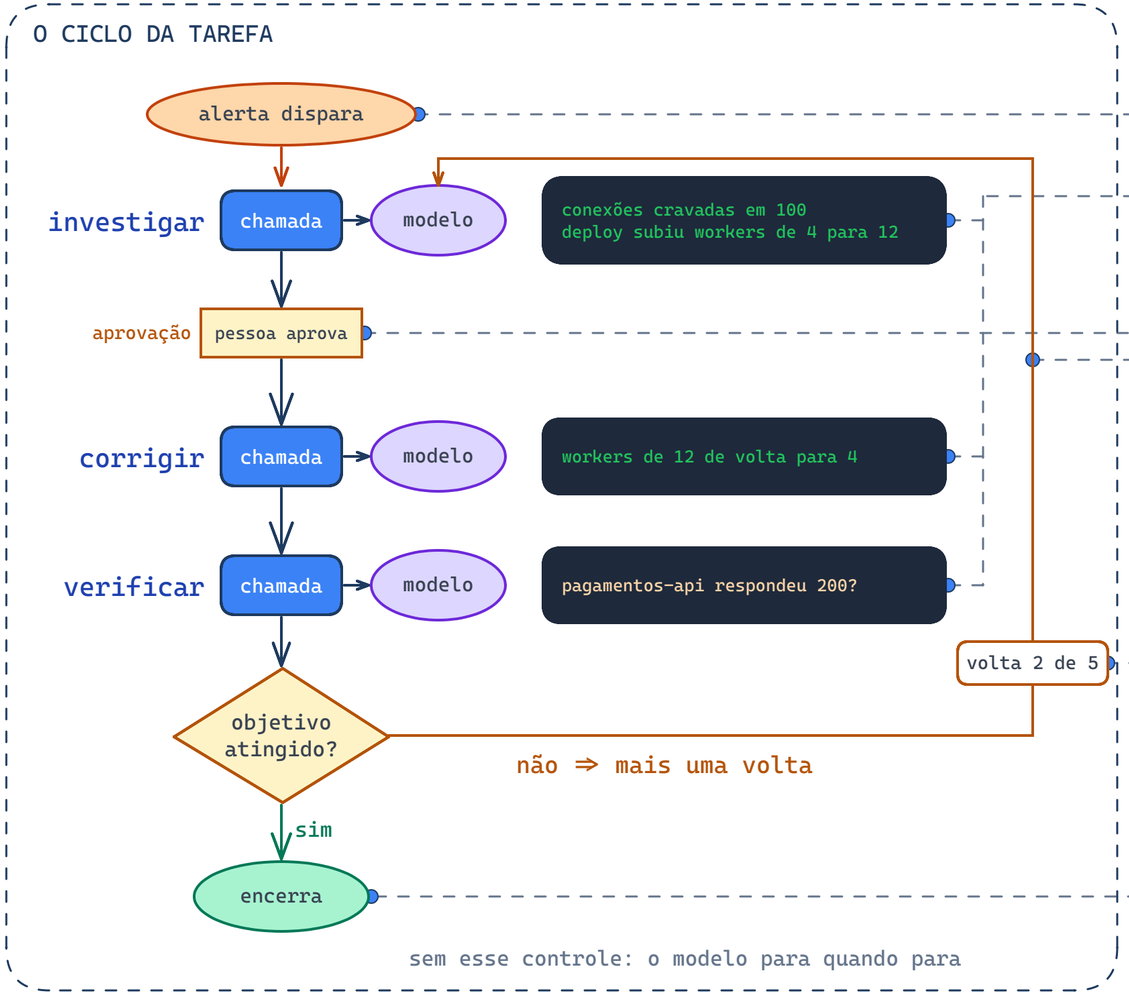
para quem ele delega uma parte

```
delega:  um agente só para ler os logs e resumir
```

Engenharia de loop

quem escreve o loop controla o ciclo, não o modelo

Loop
Harness
Contexto
Prompt



o que quem escreve o loop controla

quando começa
⇒ o alerta do pagamentos-api dispara a 1ª volta

o que cada volta faz
⇒ investigar · corrigir · verificar

onde o humano entra
⇒ aprova a correção antes de aplicar

quando dá outra volta
⇒ o objetivo ainda não foi atingido

até quantas voltas
⇒ no máximo 5, depois passa para uma pessoa — máx 5

quando para
⇒ o serviço voltou a responder 200

Engenharia de loop

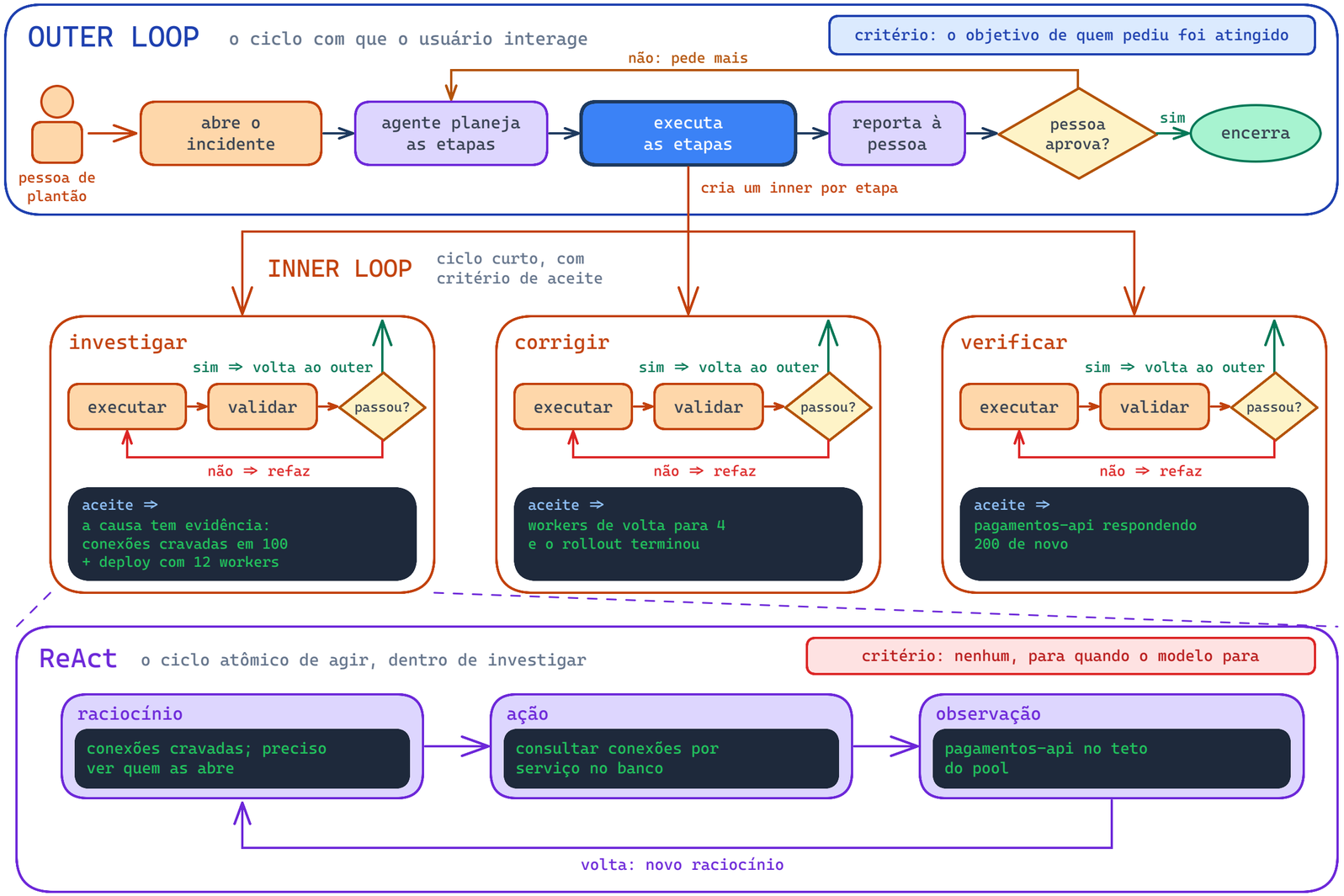
O que é
⇒ desenhar o ciclo e quem decide que acabou

O que decide
⇒ o critério de parada não o momento em que a IA para

Quanto dura
⇒ sobrevive à pessoa que escreveu

Três ciclos, três critérios

o outer cria inner loops; cada inner age em ciclos de ReAct



critério por nível

outer

objetivo de quem pediu

sem ele ⇒ roda em círculo sem fechar

inner

critério de aceite da etapa

sem ele ⇒ desiste com a etapa pela metade

ReAct

nenhum

sozinho ⇒ confunde parar com concluir

pergunta em aberto

como um agente sabe que o incidente acabou?

Quatro camadas, quatro tempos de vida

de baixo para cima, cada camada dura mais que a anterior

Loop
Harness
Contexto
Prompt

permanência		a fronteira do que você controla	o que decide	o que aparece aqui	num incidente do pagamentos-api	quando some
além da pessoa	**Loop**	o processo	o ciclo e o critério de parada	outer loop, inner loop (o outer cria o inner)	critério de parada: só fecha quando o serviço voltar a responder 200; não 200 ⇒ mais uma volta	continua depois que quem escreveu sai
projeto	**Harness** (projeto/ skills, MCP, permissões)	o projeto	o que o agente sabe e pode	skills, MCP	projeto/ skill do Dockerfile; leitura do cluster: sim; apagar pod: não	quando o projeto acaba
						sobrevive
			a sessão fecha aqui			morre
sessão	**Contexto**	uma sessão	informação certa, volume e momento	janela de contexto	janela da sessão: + projeto-pagamentos-api.md: max_connections 100; + log do horário do erro	ao fechar a sessão
mensagem	**Prompt**	uma chamada	como pedir	zero-shot, few-shot, chain-of-thought, ReAct	mensagem: > Investigue o 503 do pagamentos-api.	ao enviar a próxima mensagem

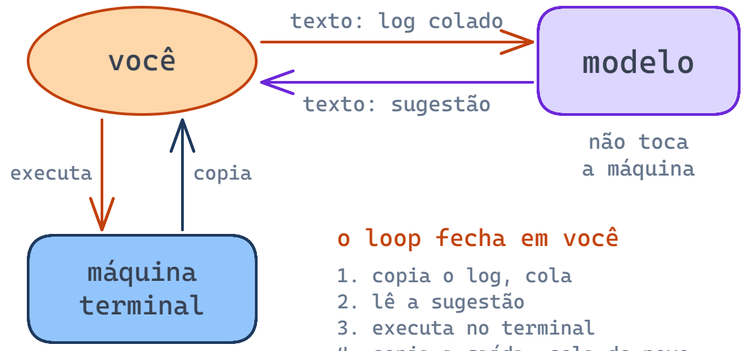
O mesmo modelo, com e sem harness

a diferença está nas peças em volta do modelo, não no modelo

chat ⇒ texto entra, texto sai
agente ⇒ lê, roda, observa

Chat

só texto entra, só texto sai



o loop fecha em você

1. copia o log, cola
2. lê a sugestão
3. executa no terminal
4. copia a saída, cola de novo

a conversa

```
você: o disco do servidor encheu, o que eu faço?
      [colou] /dev/sda1  50G  49G  1,0G  98%  /

chat: rode  du -sh /var/log/*  e me mande a saída

você: [rodou no terminal, copiou, colou de volta]
```

você é a ferramenta:
cada volta passa pela sua mão

Agente

o mesmo modelo, com peças montadas em volta: lê, roda, observa

harness

o que persiste em volta

modelo

ferramentas
ler arquivo
rodar comando
editar arquivo

instruções
o que está escrito no projeto e ele lê ao abrir

permissões
o que ele pode invocar sem pedir

diretório de trabalho
onde ele opera

alcance = consequência

máquina: o servidor, onde o agente age

```
lê    /etc/logrotate.d/app      ← ferramenta: ler arquivo
roda  df -h                     ← ferramenta + permissão
      /dev/sda1  50G  49G  98%  /
roda  du -sh /var/log/*         ← ferramenta + permissão
      38G  /var/log/app
conclui: logs da aplicação ocupam 38G
```

o que ele enxerga

- o diretório de trabalho
- a saída do comando
- o estado que ele consultou

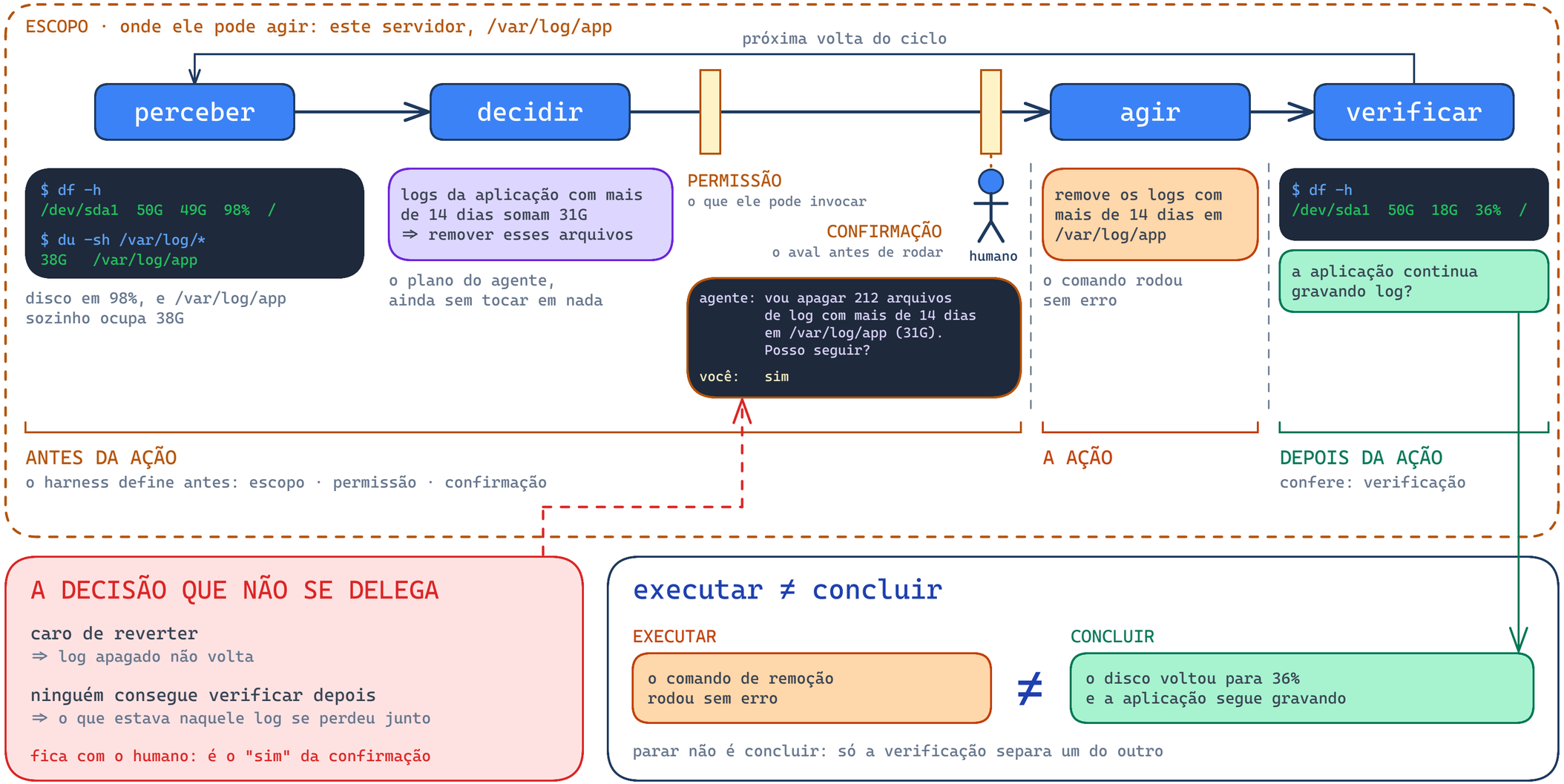
o que ele não enxerga: fora da borda

- o que nenhuma ferramenta alcança
- o que ninguém mandou ele olhar
- o que está fora da máquina

o modelo é o mesmo · o que muda é o que está montado em volta dele

Uma volta do ciclo, e onde o humano entra

caso: o disco do servidor encheu



A DECISÃO QUE NÃO SE DELEGA

caro de reverter
⇒ log apagado não volta

ninguém consegue verificar depois
⇒ o que estava naquele log se perdeu junto

fica com o humano: é o "sim" da confirmação

executar ≠ concluir

EXECUTAR

o comando de remoção rodou sem erro

≠

CONCLUIR

o disco voltou para 36% e a aplicação segue gravando

parar não é concluir: só a verificação separa um do outro

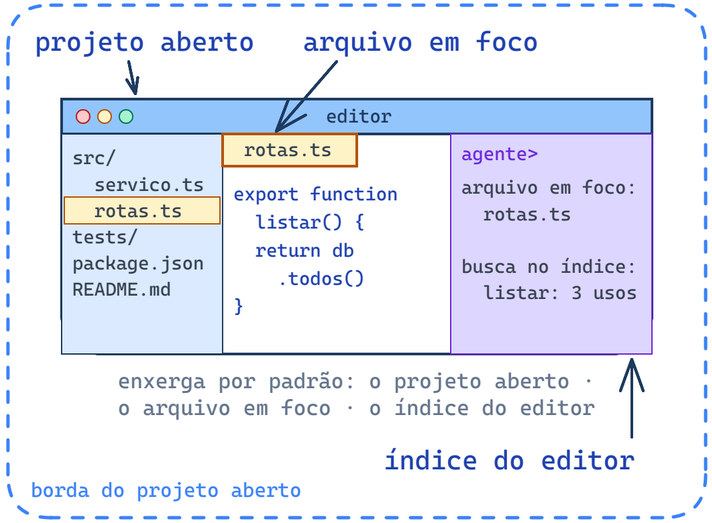
Duas superfícies, dois alcances

a diferença é o que cada superfície alcança, não qual é melhor

Agente de CLI · terminal



Agente de IDE · editor



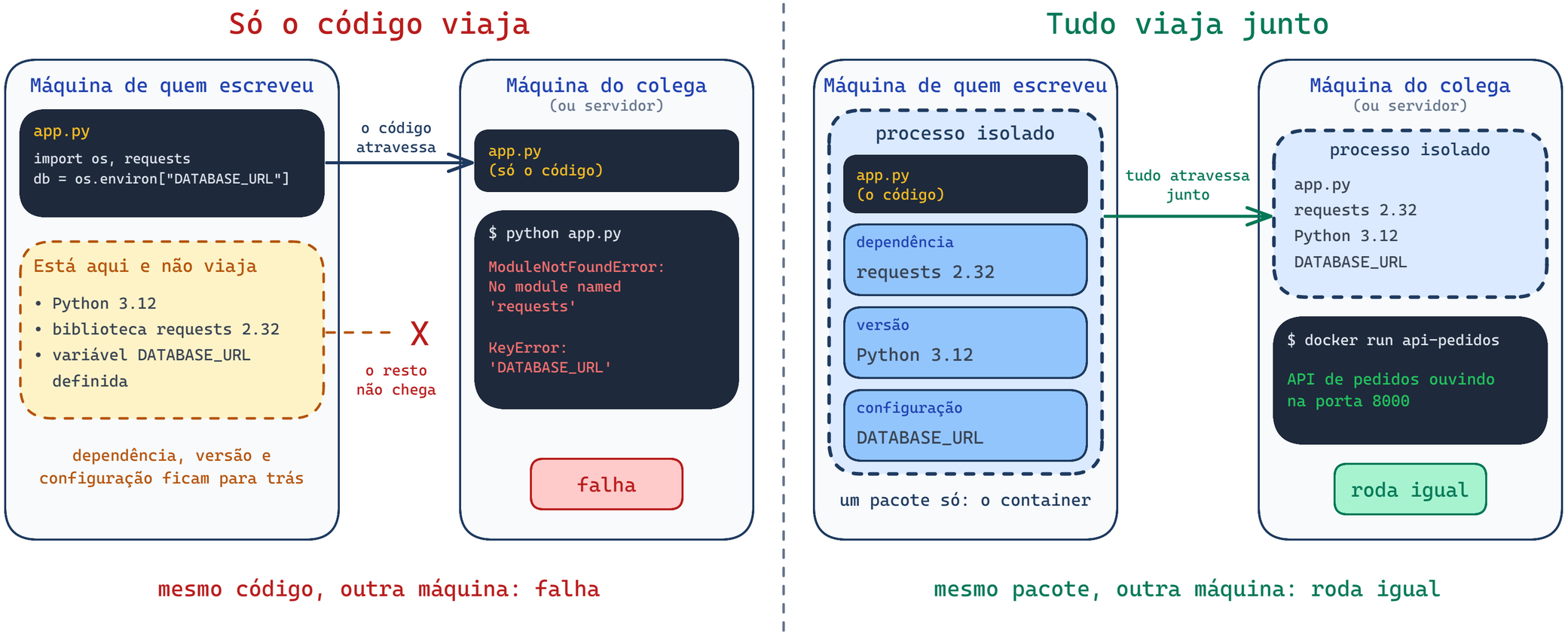
o alcance termina na borda do projeto aberto

quem opera infraestrutura não passa o dia dentro de um projeto de aplicação
⇒ o trabalho mora no shell, no pipeline, na máquina remota, no cluster

quem oferece qual superfície: confira na documentação · várias ferramentas oferecem as duas

Funciona na minha máquina

o que quebra entre máquinas não é o código, é o que fica para trás



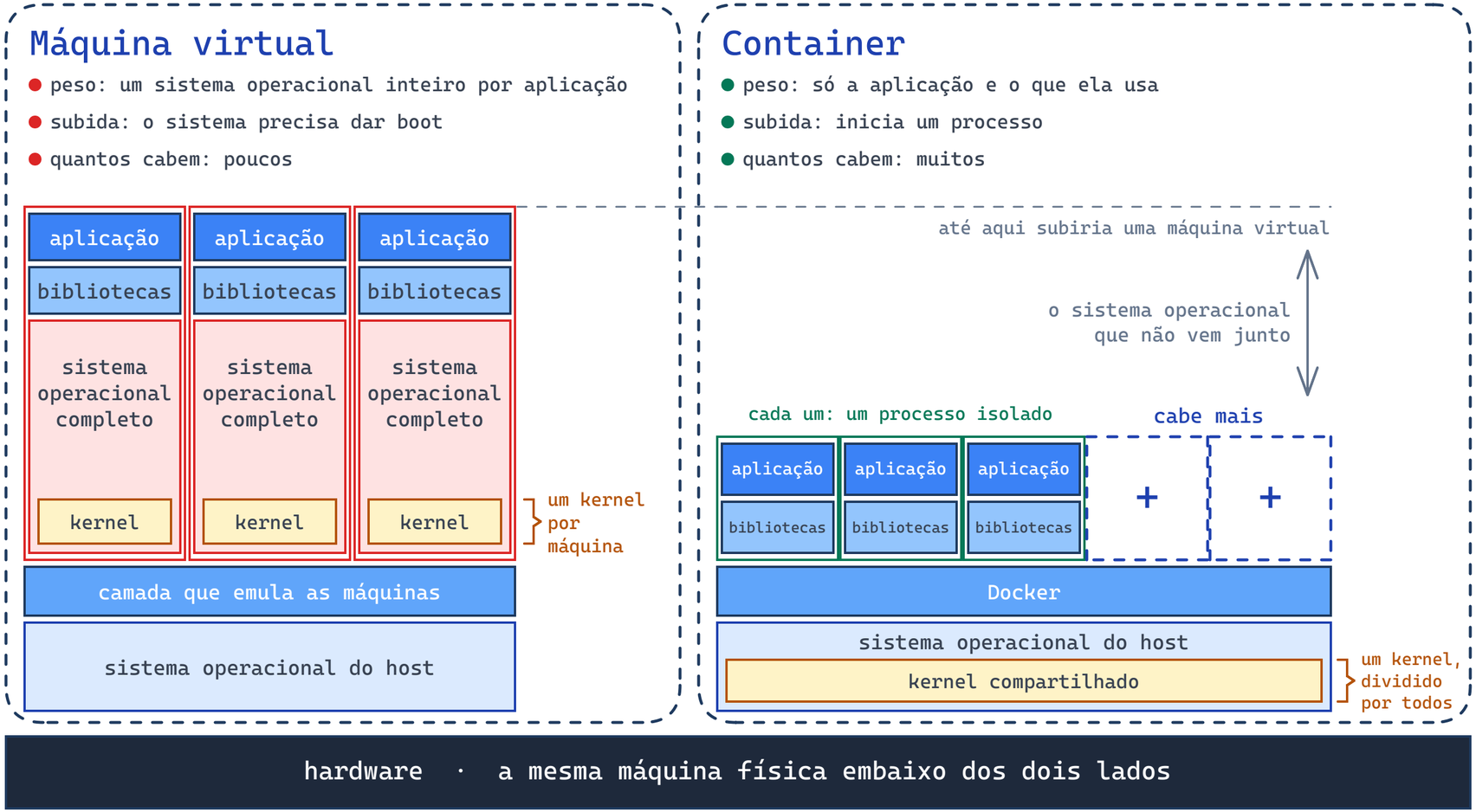
mesmo código, outra máquina: falha

mesmo pacote, outra máquina: roda igual

O que precisa viajar junto com o código ⇒ processo isolado, o propósito da ferramenta

Container × máquina virtual

Isolar o processo × emular a máquina inteira



O que muda

	Máquina virtual	Container
peso	um sistema operacional por aplicação	só a aplicação e o que usa
tempo de subida	o sistema dá boot	inicia um processo
quantos cabem	poucos	muitos

Mesmo hardware. A diferença é o que cada aplicação carrega junto.

Dockerfile, imagem e container

a receita → o artefato → as execuções

A receita

Dockerfile: arquivo de texto com instruções

```
Dockerfile
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install -r requirements.txt
5 COPY . .
  CMD ["python", "app.py"]
```

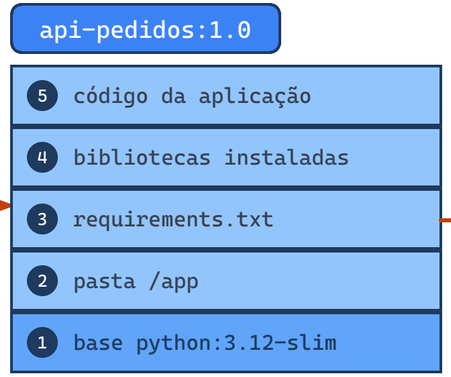
cada instrução vira uma camada

o número liga a linha à camada da imagem
CMD não cria camada: só diz como iniciar

docker build

O artefato

a imagem, construída em camadas



imutável

não muda depois de construída
empilhada de baixo para cima

docker run

As execuções

uma imagem, vários containers

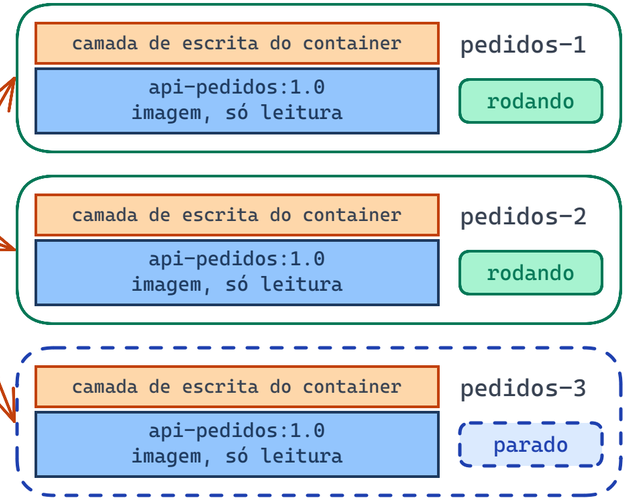


Imagem × container

parecem a mesma coisa, mas não são

	imagem	container
O que é	artefato construído	a imagem em execução
Muda?	não, é imutável	escreve na própria camada
Quantos	uma	vários, da mesma imagem
Nasce de	docker build no Dockerfile	docker run na imagem

a imagem é o molde, o container é a peça

Registry

onde as imagens são armazenadas e distribuídas



minhaempresa / **api-pedidos** : **1.0**

usuário ou organização · nome da aplicação · tag com a versão

nome · tag · versão
⇒ o endereço da imagem

a imagem só sai da máquina de quem construiu pelo registry

Compose

vários containers subindo juntos, na mesma máquina

Sem Compose

cada container subido à mão

```
$ docker run -d redis:7
$ docker run -d minhaempresa/api-pedidos:1.0
```

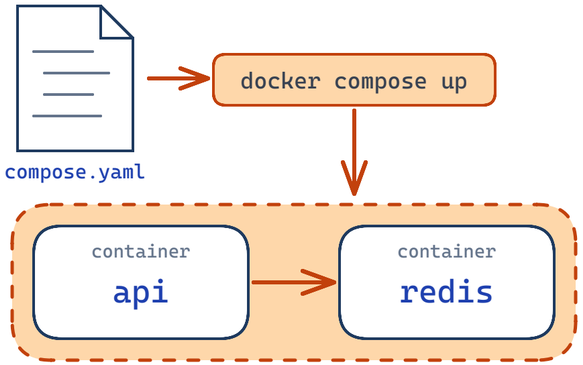
container **api** — × — container **redis**

um não sabe do outro

ordem, rede e variáveis: tudo à mão

Com Compose

um arquivo descreve o conjunto



sobem juntos e se encontram

um comando, o conjunto inteiro

O arquivo

cada amarra marcada com uma letra

```
compose.yaml
services:
  api:
    image: minhaempresa/api-pedidos:1.0
    environment:                                  A
      REDIS_URL: redis://redis:6379               B
    depends_on:                                   C
      - redis
  redis:
    image: redis:7
    volumes:                                      D
      - dados-redis:/data
volumes:
  dados-redis:
```

o que o Compose amarra

A variáveis
a configuração entra no container

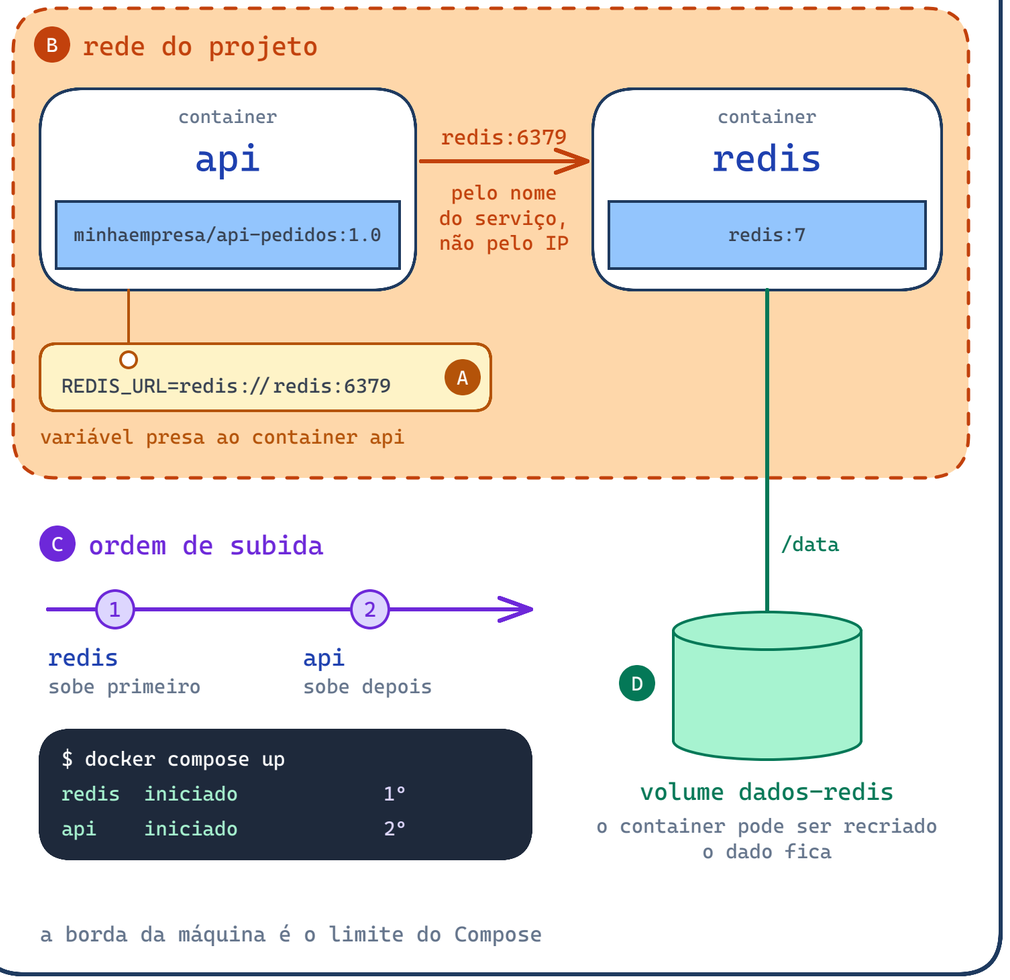
B rede
um encontra o outro pelo nome do serviço

C ordem de subida
quem precisa estar de pé antes

D volume
o dado guardado fora do container

Uma máquina

tudo acontece dentro desta borda



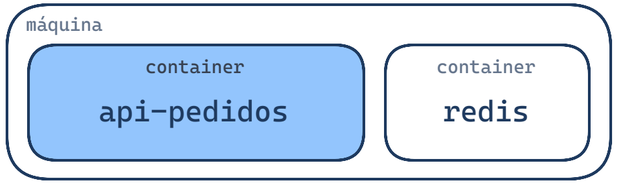
Dockerfile descreve um container · Compose descreve como vários sobem juntos e o que os amarra

Do Compose ao orquestrador

o Compose sobe o conjunto; o que vem depois, o orquestrador assume

Compose

uma máquina, um comando




```yaml
compose.yaml
services:
  api-pedidos:
    image: minhaempresa/api-pedidos:1.0
    environment:
      REDIS_HOST: redis
  redis:
    image: redis:7
```

sobe o conjunto e para por aí

fica com você, na mão:

- a carga subiu?
- o container caiu?
- outra máquina?
- quem divide as requisições?
- a configuração de cada cópia?

e quando isso vira produção?

Orquestrador

assume cinco trabalhos num cluster de várias máquinas

orquestrador

1 Escalar a aplicação

a carga subiu?

cópias: 2 → 5

cópia cópia antes

+3 cópias, mesma imagem

cópia cópia cópia cópia cópia

escala a aplicação, não a máquina

cluster

máquina 1	máquina 2	máquina 3
api-pedidos	api-pedidos	api-pedidos
api-pedidos	api-pedidos	redis

5 cópias da api-pedidos

2 Distribuir

onde cada cópia roda?

cópias → máquina 1: cópia; máquina 2: cópia; máquina 3: cópia

cada cópia numa máquina diferente

5 Gerenciar a configuração

cada cópia recebe a configuração certa?

REDIS_HOST: redis
API_TOKEN: ••••••

cópia cópia cópia

variável e segredo fora da imagem

a mesma configuração em cada cópia

4 Balancear a carga

quem divide as requisições?

requisições → endereço único: pedidos.minhaempresa.com → cópia cópia cópia

vindas de vários usuários ao mesmo tempo

um endereço, a carga dividida entre as cópias

3 Repor o que cai

um container caiu? uma máquina caiu?

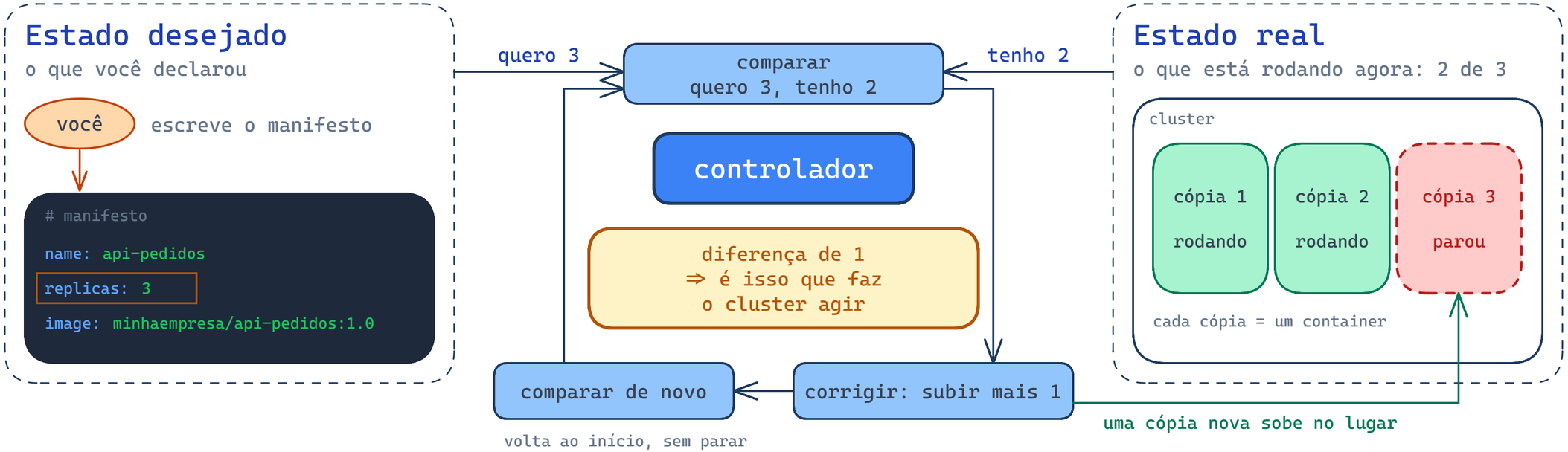
máquina 2: caiu (cópia) → máquina 3: cópia, cópia nova

o total volta a 5 cópias, sem você

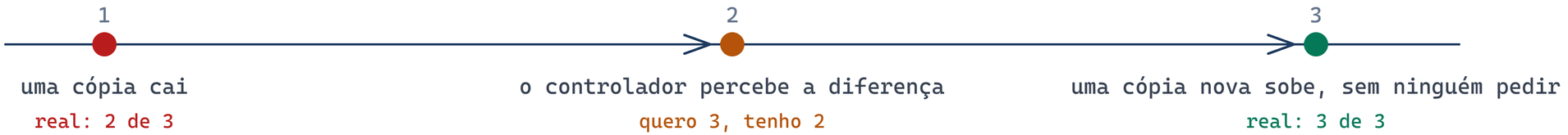
Orquestrador de containers ⇒ assumir o que vem depois de subir:
escalar, distribuir, repor, balancear e configurar, sem você fazer isso na mão

Estado desejado × estado real

dois lados e um controlador comparando os dois, sem parar

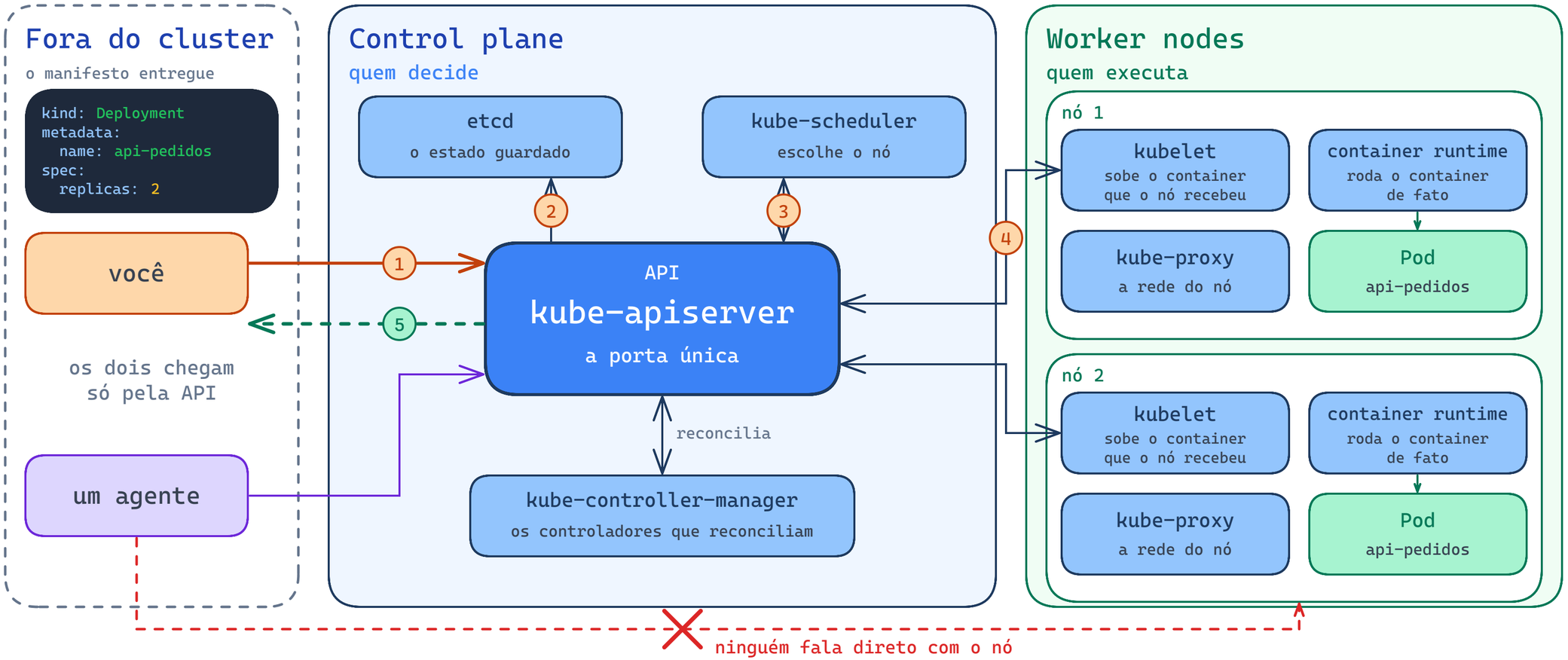


Na prática



Você declara o que quer · o cluster persegue · reconciliação ⇒ comparar e corrigir, sem ninguém pedir

A arquitetura do Kubernetes



o caminho de um pedido

1. você entrega o manifesto à API
2. a API grava no etcd
3. o kube-scheduler escolhe o nó
4. o kubelet daquele nó sobe o Pod
5. o status volta pela API

Toda conversa com o cluster passa pela API

Os objetos do deploy

quem roda as cópias da aplicação, e quem faz alguém chegar até elas

Quem roda

a cadeia de posse: cada objeto cria o de dentro

Deployment

⇒ quantas cópias e como atualizar

"quantas cópias, de qual versão?"

você declara o Deployment, não o ReplicaSet

```
nome: api-pedidos
cópias: 3
imagem: minhaempresa/api-pedidos:1.0
```

↓ cria

ReplicaSet — api-pedidos-7d9f8

⇒ mantém o número de cópias de Pod

"se uma cópia sumir, quem põe outra?"

Pod

⇒ a menor unidade que roda

"o que está rodando?"

mantém 3 →

- Pod api-pedidos-7d9f8-x2k4p — container: minhaempresa/api-pedidos:1.0 — IP 10.1.0.12
- Pod api-pedidos-7d9f8-m8q1z — container: minhaempresa/api-pedidos:1.0 — IP 10.1.0.31
- Pod api-pedidos-7d9f8-r5t7c — container: minhaempresa/api-pedidos:1.0 — IP 10.1.0.47

Pod vai e volta, o endereço muda

o ReplicaSet compara:

```
desejado: 3 cópias
rodando:  3 cópias
```

Quem encontra

um nome que não muda

Service — api-pedidos

o nome não muda

divide a carga entre as cópias

Service

⇒ o nome estável para encontrar a aplicação

"como alguém encontra as cópias?"

Tipos de Service: até onde o nome alcança

dentro do cluster | porta do nó | fora →

ClusterIP — api-pedidos:80

só dentro do cluster

quem chega: outra aplicação do cluster

NodePort — <ip-do-nó>:30080

pela porta de cada nó

quem chega: quem alcança o nó

LoadBalancer — pedidos.minhaempresa.com

por um endereço externo, entregue pelo entorno

quem chega: quem está fora

Deployment cria o ReplicaSet · ReplicaSet mantém os Pods · Pod roda o container
Service dá o nome, e o tipo dele decide até onde ele alcança

O manifesto

um arquivo, dois objetos · cada bloco cumpre um papel

api-pedidos.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-pedidos
  labels:
    app: api-pedidos
spec:
  replicas: 3
  selector:
    matchLabels:
      app: api-pedidos
  template:
    metadata:
      labels:
        app: api-pedidos
    spec:
      containers:
        - name: api-pedidos
          image: minhaempresa/api-pedidos:1.0
---
apiVersion: v1
kind: Service
metadata:
  name: api-pedidos
spec:
  selector:
    app: api-pedidos
  ports:
    - port: 80
      targetPort: 8000
```

que objeto é — apiVersion + kind

nome e labels: a identidade

metadata

o estado desejado

spec: o que você quer que exista

labels e selector:
como um objeto encontra o outro

- o Deployment acha os seus Pods
- o Service acha para onde mandar

um documento por objeto, vários no mesmo arquivo

que objeto é

a identidade

o estado desejado

o selector mira as labels dos Pods

spec × status

o mesmo objeto, dois autores

você escreve

```yaml
spec:
  replicas: 3
```

o cluster escreve

```yaml
status:
  replicas: 3
  readyReplicas: 2
```

estado desejado

Pod Pod Pod

3 réplicas

estado real

Pod Pod Pod

2 prontas · 1 a caminho

estado desejado × estado real, dentro do arquivo

você declara o spec, o cluster preenche o status
a diferença entre os dois é o que ele persegue

o cluster age até o real alcançar o desejado

Aplicar o manifesto ⇒ entregar a declaração à API · não é roteiro de passos, é o estado que o cluster passa a perseguir